LABORATORY OBSERVATION

Course:	Full Stack Web Development	**Experiment No.:**	6 (Exp 1)
Course Code:	23CM4121	**Date:**	05-10-2026
Student Name:	Nimidithalli Nandu	**Roll No.:**	A24126552100

1. TITLE

Demonstration of JavaScript Functions, ES6 Classes, and Object Instantiation

2. AIM

To understand and implement JavaScript modular functions, ES6 class syntax, constructor initialization, method declaration, and instantiation with multiple student objects.

3. OBJECTIVE

- Understand function declaration and arithmetic return values in JavaScript.
- Implement ES6 class syntax with constructors and instance properties.
- Define class methods for formatted console logging.
- Instantiate custom student objects (Nandu and Rahul) and execute methods.

4. THEORY

JavaScript functions are reusable blocks of code executed when invoked. They accept parameters and return computed values.

ECMAScript 2015 (ES6) introduced class syntax that provides a clean, object-oriented way to define blueprints for objects. The constructor method is called automatically when an object is instantiated using the new keyword.

Classes encapsulate both state (properties like name, course) and behavior (methods like introduce), promoting reusability and structured code organization.

5. ALGORITHM / PROCEDURE

1. Define a function add(a, b) taking two numerical arguments and returning their sum.

2. Call add(10, 20) and display the computed result using console.log().

3. Declare an ES6 class Student with constructor(name, course) storing properties on this.

4. Add an instance method introduce() that prints student name and course of study.

5. Instantiate student1 with ("Nandu", "CSM (AI & ML)") and student2 with ("Rahul", "CSE").

6. Call introduce() on both instances and verify the printed introduction text in the console.

6. PROGRAM

code.js

```
// Function
function add(a, b) {
    return a + b;
}

console.log(add(10, 20));


// Class
class Student {

    constructor(name, course) {
        this.name = name;
        this.course = course;
    }

    introduce() {
        console.log("I am " + this.name +
                    " studying " + this.course);
    }
}

// Creating multiple objects with student details
const student1 = new Student("Nandu", "CSM (AI & ML)");
const student2 = new Student("Rahul", "CSE");

student1.introduce();
student2.introduce();
```

7. CODE EXPLANATION

Code Element	Detailed Explanation
function add(a, b)	Declares an addition function returning the sum of arguments a and b.
class Student	Defines a modern ES6 class blueprint for student entities.
constructor(name, course)	Special constructor method initializing instance properties name and course.
introduce()	Instance method that concatenates student attributes into an introduction message.
new Student("Nandu", ...)	Instantiates student1 object with specific student name and branch.

8. TEST CASES AND EXECUTION DATA

Test Case	Action / Input	Expected Result	Actual Result	Status
TC-01	Invoke add(10, 20)	Return numerical sum 30	30 displayed in console	**Pass**
TC-02	Instantiate Student Nandu	Object created with name and course	Instance properties initialized	**Pass**
TC-03	Call student1.introduce()	Print "I am Nandu studying CSM (AI & ML)"	Introduction logged to console	**Pass**
TC-04	Call student2.introduce()	Print "I am Rahul studying CSE"	Introduction logged to console	**Pass**

9. OUTPUT

Terminal Output of js_func_classes/code.js

```
PS C:\Users\sharat\OneDrive\Desktop\fs observation tasks\task6\js_func_classes> node code.js
30
I am Nandu studying CSM (AI & ML)
I am Rahul studying CSE
PS C:\Users\sharat\OneDrive\Desktop\fs observation tasks\task6\js_func_classes>
```

10. RESULT

JavaScript functions and ES6 class syntax were successfully implemented, demonstrating object instantiation, constructor initialization, and method execution.